SecureCorp — Security Assessment Engagement (v2)

Bhaskar · Cyber Security Engineer

September 7, 2026

CONTENTS

SecureCorp — Security Assessment Engagement (v2)

Junior Cybersecurity Consultant Handbook · A Four-Week Capstone *Cybersecurity Fundamentals → Practical Security Operations* **CONFIDENTIAL — For Consultant Use Only**

> **Version 2.0** — verified against VirtualBox 7.0/7.1 · Kali 2025.x · Metasploitable2 · DVWA v1.10 · Wazuh 4.9. Re-check tool versions before starting; installer commands change. What changed from v1 and why: see `WHAT-CHANGED-v1-to-v2.md`.

Contents

---

1. Welcome Letter

SecureCorp Inc. · 142 Ashford Business Park, Suite 300 · Office of the IT Director

Dear Consultant,

We've had suspicious activity on our systems and no dedicated security team, so we've engaged your group. Assess our security posture, find the weaknesses, **investigate whether the recent activity is a genuine incident**, and recommend practical fixes.

You have four weeks. We expect a written report and a short leadership debrief. This is your first engagement of this kind — work carefully, document everything, ask when something is unclear. We will make real budget decisions from your findings.

— M. Alvarez, IT Director

---

2. Project Mission & the Security Operations Loop

You are a **Junior Cybersecurity Consultant**. You will build an isolated lab that stands in for SecureCorp's server, assess it as an attacker, then switch to the defender's chair and investigate the activity — **including some activity you did not cause.**

By the end you can: build and isolate a security lab · identify hosts/ports/services · assess a vulnerable server *and* a vulnerable web app · run controlled network and web attacks and collect evidence · ship logs to a central place · **read raw logs by hand** and then use a SIEM · **tell your own test activity apart from a real intrusion** · distinguish event / alert / incident · recommend controls and present findings to a client.

THE SECURITY OPERATIONS LOOP — THE SPINE OF THE PROJECT

#	Phase	What you do	Week
1	**Build**	create the isolated lab — attacker, target, log box	1
2	**Discover**	identify the target's hosts, ports, services — its attack surface	1–2
3	**Attack**	controlled network + web attacks against the target	2
4	**Log**	generate telemetry; ship it to the log box	2
5	**Detect**	find the activity — first **by hand in the raw logs**, then in a SIEM	3
6	**Investigate**	reconstruct what happened, as a defender who *doesn't already know*	3
7	**Assess**	risk and business impact of what you found	3–4
8	**Remediate**	recommend controls that fix the weaknesses	4
9	**Report**	communicate to SecureCorp's leadership	4

If you're ever unsure why you're doing a task, find it on this loop.

CORE · CHALLENGE · STRETCH

Everyone completes **Core**. **Challenge** and **Stretch** deepen the work without changing the main project — optional, and skipping them does not affect whether you meet the objectives.

- **Core (everyone):** build the isolated lab · asset/CIA/risk analysis · `nmap` recon + service inventory · exploit **one real service vulnerability** to a shell · the four DVWA attacks at security *low* · the three-layer credential-attack comparison (SSH / FTP / DVWA) · **manual log analysis** of your own activity · **the seeded-incident investigation** · risk assessment, report, presentation.
- **Challenge:** DVWA at security *medium* (show that filtering isn't a fix) · crack the hashes you extract · Wireshark on one captured interaction · deploy **Wazuh** and compare its detection to your manual analysis · one extra documented finding.

- **Stretch:** a Windows VM with a Wazuh agent · a **custom Wazuh rule** that auto-detects one of your web attacks · an additional vulnerable service, assessed · a more advanced network design.

---

3. Company Background — SecureCorp

SecureCorp is fictional. Treat the details below as ground truth — don't invent extra systems.

Attribute	Detail
Industry	B2B SaaS (software development)
Size	35 employees — Engineering, HR, Finance, Customer Support
Primary systems	internal website, customer portal, an **Ubuntu web server**, Windows workstations
Work model	mix of in-office and remote
Security team	none — first engagement of this kind

Key assets (as the client describes them): employee laptops/workstations · the Ubuntu web server hosting the customer portal · the customer portal web app (logins, customer data, support tickets) · the internal website · the customer + employee database · employee accounts and credentials · the internet-facing connection.

In your lab: the customer portal = **DVWA** on a **Metasploitable2** target VM (that VM = SecureCorp's production server). A separate **log box** = SecureCorp's monitoring. An optional Windows VM (Stretch) = an employee workstation.

---

4. Rules of Engagement (ROE)

Every real engagement runs under a signed ROE. Yours is below. **These rules are not optional.**

SCOPE

Authorized **only** against **the isolated lab you personally build for this project** — your own VMs, on a VirtualBox **Internal Network** with no route to any other network. Only the **assigned target IP** (`10.10.10.10`) may be assessed.

RULES

1. **Only your own isolated lab.** Never scan, probe, or attack a public host, a device on a shared/home/campus/public network, or another student's machine or account — even if invited.
2. **The lab is genuinely isolated.** Vulnerable VMs (Metasploitable2 + DVWA) sit on the Internal Network **only** — never a Bridged or NAT adapter. (Metasploitable2 ships real backdoors; on a real network it is a free foothold for an actual attacker.)
3. **Password attacks** target only accounts you created yourself, or the well-known Metasploitable2 accounts, inside the lab.
4. **Every finding is backed by evidence** — a screenshot, terminal output, or a log excerpt. Undocumented claims are not credited.
5. **Scan the single target address, not a range.** Never `nmap 10.10.10.0/24` — `nmap 10.10.10.10`.
6. If you're unsure whether something is in scope, **stop and ask** before proceeding.
7. **If you find evidence of activity you did not cause** (Week 3), that is expected — treat it as the investigation, document it, and reach a conclusion. Do not "clean it up".

TEAM STRUCTURE

Individual engagement. Each consultant builds their own lab, does their own testing, submits their own report and evidence.

---

5. How to Use This Handbook

Organized by week, then by task. Every task has the same five parts:

Part	Purpose
Objective	why this task exists, what it does for the client
Background	the context you need before starting
Tasks	what to achieve — **not** the exact commands. You research the how.
Expected Output	exactly what must be in your submission when it's done
Learning Outcome	the skill it builds

Some tasks add a **Hint** — a nudge, never a step-by-step. Templates are inline and in the appendices. **Appendix E** is your single source of truth for what to hand in.

---

6. Doing R&D — How to Research Like a Consultant

This handbook tells you *what* to achieve, not *how*. That's the point — in a real engagement no one hands you a script. You're given a goal and you research your way there. How to do that well:

- **Official source first.** For every tool (VirtualBox, Metasploitable2, DVWA, Wazuh, Nmap, Hydra, John), the project's own docs match your exact version. Read them before a blog post — outdated tutorials are the #1 reason a step "doesn't work".
- **Read the error before you search.** It usually names the problem — a missing package, permission denied, connection refused, a wrong path. Search the *exact* error text in quotes.
- **Use the tool's own help.** `man <tool>`, `<tool> --help`. Faster than hunting for the right blog, and a core habit in this field.
- **Change one thing at a time.** Change, test, observe. That's how you learn what each setting does — and if you fix it, you know why.
- **Understand before you paste.** Never run a command you can't explain in one sentence — especially anything with `sudo`. On a security team, a command you didn't understand is how incidents start.
- **Keep a research log** — what you tried, what happened, the source that helped. It becomes your report's Methodology and means you never solve the same problem twice.

- **Judge your sources.** Official docs → known community sites → dated forum posts, in that order. Check the date. When two sources disagree, the official docs win.
- **Know when to ask.** Research first is the rule; stuck for hours on the same wall is not a badge of honour. When you ask, show what you tried and the exact error — a good question is itself a research skill.

Every professional in this field looks things up daily. Treat each “I don’t know how” as the work, not a detour from it.

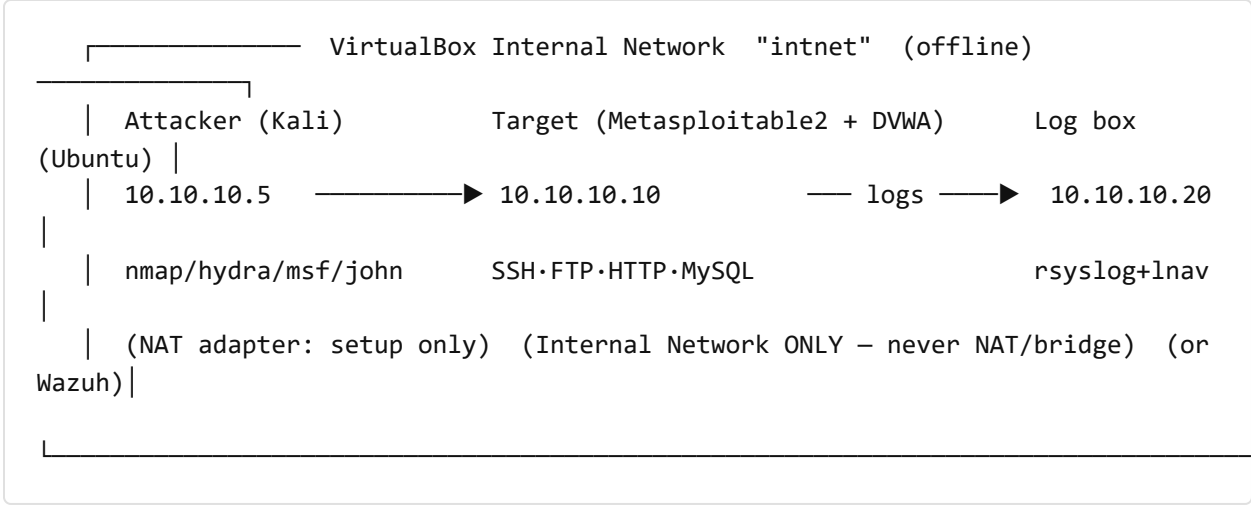
---

7. Lab Setup — the isolated lab

> **Full build guide with exact commands, static-IP config, and the troubleshooting table:** `lessons-v2/LAB-SETUP.md` §3 (“Model B — student-built isolated lab”). Read it. This section is the summary and the requirements.

WHAT YOU BUILD

Three machines on **one VirtualBox Internal Network** named `intnet`. **Nothing is bridged.** The whole lab has no route to the internet except a temporary NAT adapter on the attacker, used only to install tools and then disabled.



REQUIRED SOFTWARE

Tool	Role
VirtualBox (+ Extension Pack)	run the VMs
Attacker VM — Kali (or Ubuntu + tools)	your consulting laptop: `nmap`, `hydra`, `metasploit-framework`, `john` / `hashcat`, `wireshark`, `curl`, a browser, `seclists`
Target VM — `securecorp-target.ova` (provided)	Metasploitable2 with DVWA pre-installed and the seed script staged. *(If not provided: build DVWA on Metasploitable2 with a temporary NAT adapter, then remove it — see LAB-SETUP.md §3.3.)*
Log box VM — Ubuntu Server	receives the target's logs. **Manual path (Core):** `rsyslog` + `lnav`. **Wazuh path (Challenge / if 16 GB):** the Wazuh single-node quickstart, or a shared instructor Wazuh.

HARDWARE

- One laptop/desktop with **VT-x / AMD-V enabled in BIOS** (and Hyper-V/WSL2/Memory-Integrity **off** on Windows — they steal VT-x from VirtualBox).
- **8 GB RAM** runs the Core lab (Kali 3 GB + Metasploitable2 0.5 GB + log box 1 GB).
- **16 GB** is needed for the Wazuh path. On 8 GB, use a **shared instructor Wazuh** for the Challenge tasks, or stay on the manual path (which is Core anyway).
- 40 GB free disk.

WHY NOT WSL2?

WSL2 can't cleanly reach an isolated VirtualBox network, which is what pushes people toward the unsafe *bridged* setup that puts Metasploitable2 on their home Wi-Fi. **Use a Kali/Ubuntu VM as the attacker.** (Advanced students who insist on WSL2: it's on you to prove your target never touches a real network — and it's much harder than just using a VM.)

WAZUH, POSITIONED HONESTLY

Everyone does manual log analysis first, as its own graded step (Task 3.2) — you read the raw `auth.log`, correlate failed logins by source IP and timestamp *by hand*. That is the SOC skill. **Then** (Challenge, or Core if you have 16 GB / a shared server) you deploy Wazuh, ingest the same logs, and answer: *what did the SIEM catch that I missed, and what did I catch that it didn't?* A **custom Wazuh rule** is Stretch.

---

8. Time Budget & Readiness Gates

Budget ~35–45 hours across four weeks. If you're 5+ hours over on Week 1, tell your instructor and switch to the shared-Wazuh / pre-built-OVA path — don't let lab-building eat the engagement.

Week	Realistic hours (beginner, with normal snags)
1 — build + understand	10–14 (budget a weekend)
2 — Red Team	10–12
3 — Blue Team + the seeded incident	8–10
4 — recommendations + debrief	6–8

READINESS GATES — DO NOT START THE NEXT WEEK UNTIL EVERY BOX PASSES

End of Week 1 gate

```
☐  attacker `ping 10.10.10.10` succeeds from the intnet adapter
☐  DVWA loads at http://10.10.10.10/dvwa ; Setup/Reset done ; security = low
☐  `nmap -sV 10.10.10.10` returns services from the attacker
☐  the target's auth.log is arriving on the log box  (`lnav` shows recent lines)
☐  a deliberate FAILED SSH login from the attacker produces a VISIBLE event on
the log box
     — the hard gate. If you can't see one failed login now, you can't
investigate six
     attacks + a seeded incident in Week 3.
☐  snapshots taken: attacker "w1-clean", target "w1-clean", log box "w1-clean"
```

End of Week 2 gate *(bring your Attack Log to your instructor for a 5-minute check before Week 3)*

```
☐  service inventory complete (every open port, service, version)
☐  one real service vulnerability exploited to a shell, with evidence
☐  four DVWA attacks done at security low, each with a timestamped screenshot + source IP
☐  SSH + FTP + DVWA brute force each done, with start/end times recorded
☐  target restored to the "w1-clean" snapshot, then a fresh "w2-attacked" snapshot taken
```

---

9. Week 1 — Build & Understand

Stage: Scoping & Discovery. You are not testing anything yet. You build the isolated lab and build your understanding of SecureCorp as a business.

TASK 1.1 — BUILD THE ISOLATED LAB

Objective: stand up the environment you'll assess for the rest of the engagement — safely.

Background: SecureCorp's portal = DVWA on Metasploitable2. Your consulting laptop = the attacker VM. See §7 and `LAB-SETUP.md` §3.

Tasks

- Install VirtualBox; import the attacker VM, the target VM (`securecorp-target.ova`), and create the log box VM.
- Put **all three** on the same **Internal Network** `intnet`. Give each a static IP (`.5`, `.10`, `.20`). The attacker gets a second **NAT** adapter *for tool install only* — disable it once tools are in.
- Confirm the attacker can reach the target (`ping`, then the DVWA page in a browser).
- On Metasploitable2, log in at the console with the documented default credentials, then **research how to change the default password** and do it. Do not leave defaults for the engagement.

Expected Output

- Screenshots: each VM running, its IP visible; DVWA loaded in the attacker's browser.
- A **network diagram** — all three machines, their IPs, the Internal Network, and the note "no bridged/NAT adapter on the target". Redraw it with your real values.

Learning Outcome: how an assessment environment is built and, critically, **isolated** from everything else.

Hint: take the setup screenshots now — you want a clean "before" state.

TASK 1.2 — ASSET IDENTIFICATION

Objective: identify everything at SecureCorp with value.

Tasks: from §3, list every asset; classify each (hardware / software / data / personnel / connectivity); note who owns or is affected by each.

Expected Output: a completed Asset Inventory table.

Asset	Type	Owner / Dept	Why it matters

TASK 1.3 — CIA ANALYSIS

Objective: apply Confidentiality / Integrity / Availability to real assets.

Tasks: for each major asset, explain why C, I, and A each matter for it. Cover ≥ 5 assets.

Asset	Confidentiality	Integrity	Availability

Hint: *Customer Database — C: data must stay private. I: must not be altered without authorization. A: employees must reach it when needed.*

TASK 1.4 — THREAT & RISK ASSESSMENT

Objective: connect threat categories to SecureCorp's assets and business impact.

Tasks: list plausible threats (malware, SQLi, brute force, insider, phishing, …); for each, name the affected asset, the business impact, and a control that reduces the risk.

Asset	Threat	Vulnerability	Impact	Risk Rating	Recommendation

TASK 1.5 — CENTRALIZED LOGGING

Objective: get the target's logs to one place so Week 3 works from real telemetry.

Background: a real security team never runs its monitoring on the box it monitors — hence the separate log box. **Core = the manual path** (`rsyslog` receiving `auth.log` + the Apache log; investigate with `lnav`). **Challenge = Wazuh** (single-node, or a shared instructor server if you're on 8 GB).

Tasks

- On the log box: set up the `rsyslog` receiver (`LAB-SETUP.md` §3.4).
- On the target: forward the **Linux auth log** and the **Apache/DVWA access log** to `10.10.10.20`.
- Confirm lines are arriving (`lnav /var/log/lab/...`).
- **Trigger your first event:** from the attacker, make ~5 failed SSH logins against the target. Confirm they appear on the log box. *This proves the pipeline before Week 2.*
- *(Challenge)* deploy Wazuh, enroll the target agent, and confirm the same events appear as collected events and/or alerts.

Expected Output

- A screenshot of the log box showing the target's `auth.log` lines arriving with a recent timestamp.
- The failed-SSH-login events, visible.
- *(Challenge)* a Wazuh dashboard screenshot with the target agent Active.

Learning Outcome: how centralized logging works, and why you verify coverage *before* you rely on it.

WEEK 1 DELIVERABLES

```
☐  Lab setup screenshots + network diagram (isolation noted)
☐  Asset Inventory · CIA Analysis · Risk Assessment tables
☐  Log-box screenshot: auth.log arriving + the failed-SSH events visible
☐  (Challenge) Wazuh dashboard screenshot
☐  Progress note (3–4 bullets): done / blockers
☐  End-of-Week-1 readiness gate (§8) — all boxes passed
```

---

10. Week 2 — Red Team Assessment

Stage: Assessment. Think like an attacker to find weaknesses before a real one does. Not advanced pentesting — the required, well-understood simulations, documented precisely enough for Week 3.

Run every attack **from the attacker VM against** `10.10.10.10` **over the network** — never locally on the target. Stay inside the ROE.

THE METHOD — RECON BEFORE EXPLOITATION

1. **Reconnaissance** — “what can I see?” — the target’s IP and which ports respond.
2. **Enumeration** — “what’s running?” — the service and version behind each port.
3. **Assessment** — “what looks weak?” — decide what to test.
4. **Attack simulation** — “can I demonstrate it?” — the controlled attack.
5. **Evidence** — “can I prove it?” — screenshot + output + timestamp, before moving on.

Recon and enumeration are **not** attacks — they’re how you decide what to attack.

BEFORE YOU ATTACK — THE BASELINE

Record what “normal” looks like, labelled **baseline**: the target’s IP / open ports / services; DVWA loading normally; FTP and SSH reachable; the log box quiet. In Week 3 you compare against this.

TASK 2.1 — NETWORK DISCOVERY

Objective: how an attacker gathers information before touching anything else.

Tasks: from the attacker, identify the target's IP, open ports, running services and versions, and what authentication or application attack surface each one represents. Document everything.

Expected Output: recorded scan output (IP, ports, services, versions) + a short note on what each open port suggests.

Hint: `nmap -sV -sC` gets you services + versions + a first pass of safe checks. Save the output (`-oA`).

TASK 2.2 — UNDERSTAND THE TWO ATTACK SURFACES

Objective: the target is a **server** (network services on ports) that also **hosts a web app** (DVWA). Both are attackable, differently.

```
Metasploitable2 (target server)
├─ Network services
│    ├─ SSH  → SSH brute force (2.8)
│    ├─ FTP  → FTP brute force (2.7)
│    └─ other exposed services → one real exploit (2.3)
└─ Web service (HTTP)
     └─ DVWA → Brute Force (2.4), SQLi (2.5), XSS (2.6), Command Injection
(2.6b)
```

Tasks: browse DVWA, identify its login page and each module you'll use; note what each does and what input it accepts. Map the server's network services separately.

Expected Output: an application map (each DVWA module + what it does) and a network-service list.

TASK 2.3 — EXPLOIT ONE REAL SERVICE VULNERABILITY *(NEW IN V2)*

Objective: go from a scan finding to **code execution** — the thing a real attacker does.

Background: Metasploitable2 runs several services with known, well-documented vulnerabilities (e.g. the backdoored vsftpd, the Samba “usermap” issue, distccd, UnrealIRCd). Pick **one** from your Task 2.1 results.

Tasks

- From your scan output, pick a service + version with a known exploit. Confirm it with `searchsploit <service> <version>` or the NVD.
- Get a shell on the target — via a Metasploit module **or** by running a public proof-of-concept you can explain.
- From the shell, prove it: `id`, `hostname`, and read one file that proves access (e.g. `/etc/passwd`).
- *(Challenge)* enumerate for privilege escalation (`sudo -l`, `find / -perm -4000`); grab `/etc/shadow`; crack one hash with `john` / `hashcat`.

Expected Output: the Standard Attack Evidence entry — the service, the CVE, the tool/PoC, the command, and a screenshot of the shell showing `id` and the proof file. Record the exact time and the source IP.

Learning Outcome: how a version banner becomes a CVE becomes a shell — and why patching and least privilege matter.

Hint: `searchsploit vsftpd 2.3.4` is a one-liner. Keep it controlled — a shell and one proof file, nothing destructive.

TASK 2.4 — DVWA BRUTE FORCE

Objective: a credential attack against the portal login; generate application evidence.

Tasks: DVWA security **low**; from the attacker, brute-force DVWA’s Brute Force login with a small realistic password list. Record attempts, timing, result, **start/end time, source IP**. *(Challenge: set security to **medium** and show a smarter approach still works — filtering isn’t a fix.)*

Expected Output: terminal output/log with timestamps + the result. Start/end time + source IP recorded.

Hint: Hydra automates the HTTP form. Keep the wordlist small — observe the behaviour, don’t run for hours.

TASK 2.5 — SQL INJECTION

Objective: extract data the portal should protect.

Tasks: DVWA SQLi module, security **low**. Find the injectable field; craft a payload that returns data you shouldn't see (extra user records); record the exact payload and response. *(Challenge:* `UNION SELECT user, password FROM users` *→ then crack those hashes with* `john`*.)*

Expected Output: screenshot of the payload + the disclosed data, with a timestamp and source IP.

Hint: start with a single quote to see how the query breaks.

TASK 2.6 — XSS AND COMMAND INJECTION

Objective: show unsanitized input running in a browser (XSS) and on the OS (command injection).

Tasks:

- **XSS** (DVWA XSS module, low): inject a script payload; confirm it executes (an alert box is enough); record payload, location, result.
- **Command Injection** (DVWA module, low): find the field that passes input to a system command; chain a **harmless** extra command (list files); record payload + output. *(Challenge: turn command injection into a reverse shell back to the attacker.)*

Expected Output: a screenshot for each — payload + result — with timestamps and source IP.

Hint: for command injection, chain a harmless command like `; ls`, never anything destructive.

TASK 2.7 — FTP BRUTE FORCE

Objective: assess a network-service authentication mechanism separate from SSH and DVWA, and see how FTP login activity looks in logs.

Background: the target exposes FTP on 21. Metasploitable2 accounts that exist for this: `msfadmin`, `user`, `service`, `postgres`, `sys`. Use a small list.

Tasks: confirm the FTP service from Task 2.1; from the attacker, run a controlled brute-force against the FTP login with a small list; record source IP, target IP, port, username, attempts, start/end time, result. **Identify which target-side log should contain this** — you need it in Week 3.

Expected Output: screenshot/output of the FTP attempts, timestamped, + the expected log source.

Hint: Hydra has an FTP module — research the flag. Small attempt count.

TASK 2.8 — SSH BRUTE FORCE

Objective: brute-force the SSH login of an account **you create**, and see how it appears in the logs (and, if you deployed Wazuh, whether it auto-alerts).

Tasks: on the target, create a low-privilege user with a **deliberately weak** password; from the attacker, brute-force that user's SSH login; record target user, attempts, timestamp, result. Then find it — in the raw `auth.log` on the log box, and (Challenge) check whether Wazuh alerted.

Expected Output: screenshot of the `auth.log` events (and/or the Wazuh alert), timestamped, with detection status recorded as **Manual / Automatic / Not Visible**.

THE POINT OF THREE BRUTE FORCES

SSH, FTP, and DVWA are **the same technique** (credential guessing) at **three different layers**. In Week 3 you'll see they produce **three different footprints** in the logs. That comparison is the lesson — not doing the same attack three times.

ATTACK LOG (FILL FOR EVERY REQUIRED ACTIVITY)

Attack	Target / Module	Method	Start–End	Source IP	Result / Evidence	Detection (Wk 3)

WEEK 2 DELIVERABLES

```
☐  Recon notes: IP, ports, services, versions (+ saved nmap output)
☐  Application map + network-service list
☐  Task 2.3: one real exploit to a shell, with evidence
☐  Attack Log covering 2.3–2.8, each with evidence + timestamp + source IP
☐  Progress note (3–4 bullets)
☐  End-of-Week-2 readiness gate (§8) — checked with your instructor
```

Keep everything — especially every timestamp and source IP.

---

11. Week 3 — Blue Team: Investigation & Detection

Stage: Detection & Investigation. SecureCorp's monitoring flagged unusual activity on the customer portal. **Some of it is your Week 2 testing. Some of it is not.** Put on the Blue Team hat and investigate — as if you had no prior knowledge.

> **Before Week 3:** your instructor runs `seed-incident.sh` against your target (or hands you a "collected evidence" bundle). This plants activity **you did not cause** — the thing you're actually investigating. Its details are randomised per student.

YOU ARE NOW CHANGING ROLES

Week 2: *"I did this attack."* → Week 3: *"I'm a SOC analyst. What happened?"* The discipline of setting aside what you already know and letting the evidence speak is the core skill of the week.

EVENT → ALERT → INVESTIGATION → INCIDENT

- **Event** — one logged activity (a login attempt, an HTTP request). Most are harmless.
- **Alert** — an event or pattern a rule flagged for a human (many failed logins in a row).
- **Investigation** — the analyst's work of deciding what really happened.
- **Incident** — a confirmed security event with real impact that requires a response (confirmed unauthorized access).

Single failed login → event. Repeated failed logins → alert. Confirmed unauthorized access → incident. Part of your job is deciding, for each activity, where it sits.

TASK 3.1 — CONFIRM COVERAGE

Objective: confirm your monitoring captured what happened before you rely on it.

Tasks: on the log box, confirm recent lines are arriving from the target; confirm both the `auth.log` and the Apache/DVWA log are present. *(Wazuh path: agent Active, both log sources ingested.)*

Expected Output: a screenshot showing current log ingestion (a recent timestamp).

TASK 3.2 — MANUAL INVESTIGATION OF THE REQUIRED ACTIVITIES *(CORE — DO THIS BY HAND)*

Objective: locate the evidence of each Week 2 activity in the **raw logs**, the way a SOC analyst triages — before any SIEM does it for you.

Tasks

- Using `lnav` / `grep` / `journalctl` on the log box, find the log evidence for each Week 2 activity: SSH brute force, FTP brute force, DVWA brute force, SQLi, XSS, command injection, and the Nmap scan.
- Search by **source IP** or by a distinctive part of your **payload** to narrow fast.
- For **each** activity, record: **detection status** (in the Core manual path, everything is “Manual” unless you also did the Wazuh Challenge), **log source**, your **search method**, and a short note of **how the evidence connects to the attack**. If evidence for something cannot be found, record **”Not Visible”** and explain the logging limitation — *a missing log is a valid, gradeable finding.*
- For each activity, answer the **same ten analyst questions** (builds a repeatable habit):
    1. What happened? 2. When? 3. Which system? 4. Which source? 5. Source IP?
    2. What account/app was targeted? 7. Auto-detected or manual search? 8. What evidence?
    3. Potential impact? 10. What should SecureCorp do next?

Expected Output: one short detection note per activity, each with a supporting log screenshot, plus the ten answers.

Learning Outcome: what a raw log actually contains, and how the *same* technique (three brute forces) leaves *three different footprints*.

TASK 3.3 — THE SEEDED INCIDENT *(NEW IN V2 — THIS IS THE REAL INVESTIGATION)*

Objective: find the activity you **did not** cause, and decide whether SecureCorp has a genuine incident.

Background: among the logs are events that don't match anything in your Attack Log. That's the point — a real analyst arrives to a pile of activity and has to separate their own noise (if any) from a real signal.

Tasks

- Go through the target's logs (`auth.log`, Apache log, cron, `/tmp`, outbound connections on the log box) and identify every event that is **not** in your Week 2 Attack Log.
- Build the **timeline** of that activity: what, when, from where, and what it led to.
- Determine: was there a **successful** unauthorized access? Was anything **installed** (persistence)? Was data **taken** (exfiltration)?
- Classify it on the Event → Alert → Investigation → Incident scale, with justification.
- Recommend **immediate actions** SecureCorp should take.

Expected Output: a completed **Incident Report** (Appendix B), including:

- the timeline of the seeded activity,
- the IOCs you identified (source IP, filenames, cron path, C2/exfil host),
- your classification (event / alert / incident) with the evidence for it,
- immediate recommended actions,
- and — explicitly — **which events were yours (Week 2) and which were not**, and how you told them apart.

Learning Outcome: the core SOC skill — evidence-based investigation, and separating authorized testing from a real intrusion.

Hint: your own activity all comes from `10.10.10.5` (your attacker). Anything from a *different* internal IP, or at a time you weren't working, deserves a hard look.

TASK 3.4 — DETECTION COVERAGE

After the investigation, answer in your detection notes: **What was detectable in the logs you had? What was collected but easy to miss? What evidence, if any, was missing? What would you improve?** This is **SIEM tuning** in plain terms — improving detection so important activity doesn't go unnoticed.

(Challenge: deploy Wazuh, replay the same log data, and add a column — did Wazuh auto-alert on each? Compare its coverage to your manual analysis. Stretch: write one custom rule that auto-detects your DVWA command-injection attack.)

WEEK 3 DELIVERABLES

```
☐  Coverage confirmation screenshot (Task 3.1)
☐  One detection note per Week 2 activity + the ten analyst answers (Task 3.2)
☐  Incident Report for the SEEDED activity: timeline, IOCs, classification,
actions,
     and which events were yours vs not (Task 3.3)
☐  Detection coverage notes (Task 3.4)
☐  (Challenge) Wazuh comparison column  (Stretch) one custom rule
☐  Progress note (3–4 bullets)
```

---

12. Week 4 — Recommendations & Client Debrief

Stage: Remediation & Reporting. Turn three weeks of technical work into a plan SecureCorp's leadership can act on.

TASK 4.1 — SECURITY RECOMMENDATIONS

Tasks: recommend improvements across password policy, MFA, RBAC, firewall config, patch management, logging/monitoring, backup, segmentation, and user awareness. **Tie each to a specific finding** from Weeks 2–3 where possible. Explain why each matters.

Area	Current state (evidence)	Recommendation	Priority

TASK 4.2 — EXECUTIVE SUMMARY

Tasks: one page — overall posture, the most critical findings, the highest business risks, your top recommendations. **A non-technical manager must understand it** with none of the Week 2–3 detail. Do **not** write it in the same technical language as the rest of the report.

TASK 4.3 — FINAL CLIENT DEBRIEF

Tasks: a 10–15 minute presentation — posture, key findings, evidence (a live demo is optional), recommendations, lessons learned. Be ready for follow-up questions from “SecureCorp leadership” (your instructor / peers).

Expected Output: a slide deck + the delivered presentation.

Appendix A — Master Report Template

1. Cover page — engagement title, your name, date
2. Executive Summary (4.2)
3. Scope & Rules of Engagement (§4)
4. Asset Inventory & CIA Analysis (1.2–1.3)
5. Risk Assessment (1.4)
6. Methodology — discovery, testing, investigation, your logging setup, and **whether you used the manual path or Wazuh** (1.5, plus your research log)
7. Attack Simulation Findings — the Attack Log for 2.3–2.8
8. Detection Coverage & Investigation — Task 3.2 notes, **the seeded-incident Incident Report (3.3)**, detection coverage (3.4)
9. Recommendations (4.1)
10. Appendices — raw `nmap` output, full log excerpts, extra screenshots, your research log

---

Appendix B — Evidence & Finding Templates

Evidence rules: full window (URL bar / prompt visible), not a crop · descriptive filenames (`W2-T2.5-sqli-payload.png`) · terminal output saved as text · log excerpts with surrounding context, the key line highlighted · note automatic vs manual for every SIEM finding.

File naming: `W<week>-T<task>-<description>.<ext>` — e.g. `W1-T1.1-network-diagram.png`, `W2-T2.3-vsftpd-shell.png`, `W2-T2.5-sqli.png`, `W3-T3.3-incident-timeline.png`.

STANDARD ATTACK EVIDENCE ENTRY (ONE PER ATTACK)

```
Attack ID · Target · Source IP · Target IP · Port · Service/App · Attack type · Tool/PoC
Payload / method · Start time · End time · Result · Evidence (filenames)
Log evidence (file + line) · Detection status (Automatic / Manual / Not Visible)
Log source · (Wazuh) rule ID + level · Search method / filter
Analyst explanation — how the evidence supports the attack
```

STANDARD FINDING ENTRY (ONE PER WEAKNESS)

```
Title (e.g. "Weak SSH authentication") · Affected asset · Affected service
Evidence (screenshot / log ref) · Attack demonstrated
Impact — what could happen to SecureCorp if exploited
Risk rating — Low / Moderate / High / Critical
Detection — how it appeared (alert / manual / not visible)
Recommendation — the control that fixes it
```

INCIDENT REPORT (TASK 3.3)

```
Summary (3–4 sentences a manager understands)
Timeline:
  Time | Event | Source (log) | Evidence / notes
IOCs: source IP(s) · filenames · persistence path · C2 / exfil host
Which events were MINE (Week 2) vs NOT — and how I told them apart
Classification (event / alert / incident) — with the evidence
Confirmed impact: successful access? persistence installed? data taken?
Immediate recommended actions
```

Appendix C — Marking Rubric (mapped to deliverables)

Total: 100 marks.

#	Criterion	Deliverables it covers	Marks
1	Lab build & **isolation**	1.1 (network diagram shows an isolated Internal Network; no bridged target), the Week-1 gate	8
2	Centralized logging	1.5 (logs arriving; failed-SSH events visible)	6
3	Asset / CIA / Risk analysis	1.2, 1.3, 1.4	10
4	Recon & service inventory	2.1, 2.2	8
5	Real exploitation	2.3 (shell + proof; +Challenge: priv-esc / cracked hash)	10
6	Web & credential attacks	2.4, 2.5, 2.6, 2.7, 2.8 — each with evidence + timestamp + source IP	14
7	Manual investigation	3.2 — detection note + ten answers per activity	12
8	**The seeded incident**	3.3 — timeline, IOCs, classification, “mine vs not”, actions	14
9	Detection coverage / tuning	3.4 (+Challenge: Wazuh comparison; +Stretch: custom rule)	4
10	Recommendations	4.1 — tied to findings, prioritized	8
11	Report quality & Executive Summary	Appendix A structure; 4.2 written for a non-technical reader	6
12	Presentation & communication	4.3	6
—	**Research log** (methodology evidence)	Appendix A §6, §10	(folded into 7, 8, 11)

Challenge/Stretch items add depth within their criterion — they do not add marks beyond the maximum, and skipping them cannot fail you if the Core work is solid.

Appendix D — Common Mistakes

- A finding with no screenshot / output / log excerpt.
- A finding with no recommended fix.
- ”It’s a risk” with no explanation of the impact.
- No standard terminology (CVE, CVSS, CIA, IOC) where it belongs.
- Findings scattered instead of following Appendix A.
- Treating Week 3 as unrelated to Week 2 — you’re investigating your own attacks *plus* the seeded activity.
- **Not recording timestamps and source IPs in Week 2** — makes Week 3 nearly impossible.
- **Not identifying which Week-3 events were yours vs the seeded incident** — that comparison is the point of Task 3.3.
- The Executive Summary written in the same technical language as the report.
- **A bridged adapter on the target.** This fails criterion 1 and breaks the ROE.

Appendix E — Submission Checklist

Your single source of truth. Submit as one package.

```
☐  Network diagram (isolation shown) + application map
☐  Asset Inventory · CIA Analysis · Risk Assessment
☐  Log-box screenshot (auth.log arriving; failed-SSH events)  [+ Wazuh
screenshot if Challenge]
☐  Recon output + service inventory
☐  Attack Log for 2.3–2.8 — each with evidence, timestamp, source IP
☐  Task 2.3 real-exploit evidence (shell + proof file)
☐  Detection note + ten analyst answers per Week-2 activity (3.2)
☐  Incident Report for the seeded activity (3.3) — timeline, IOCs,
classification,
     "mine vs not", actions
☐  Detection coverage notes (3.4)
☐  Security Recommendations (tied to findings)
☐  Executive Summary (one page, non-technical)
☐  Final Presentation
☐  Research log
☐  All evidence files named per Appendix B
☐  Both readiness gates (Week 1, Week 2) recorded as passed
```

Isolated-lab build detail, offline bundle, and troubleshooting: `lessons-v2/LAB-SETUP.md` *§3. Seeded-incident script:* `seed-incident.sh` *(instructor-run).*